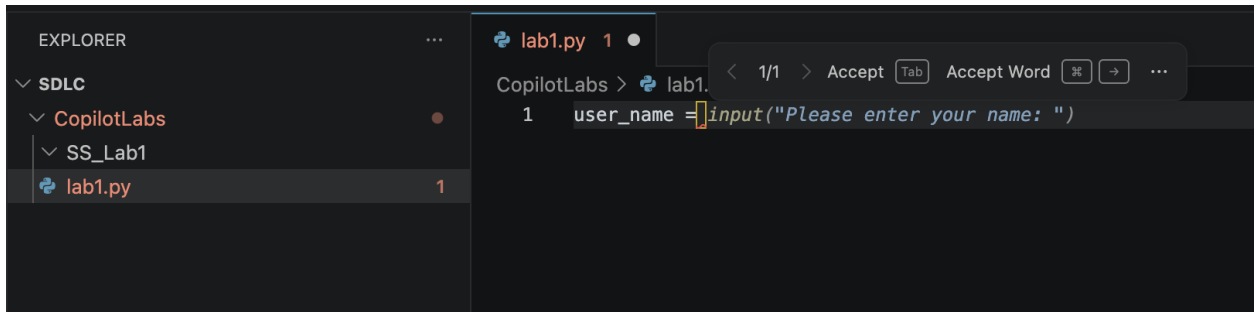


GitHub Copilot Lab 1

Step 1: Test Inline Variable Suggestion:



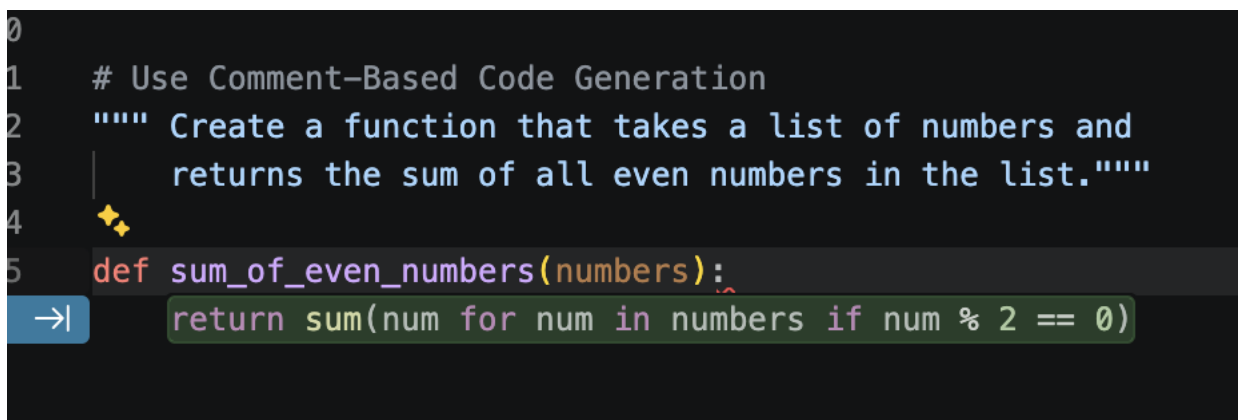
The screenshot shows the VS Code interface with the Explorer on the left and the editor on the right. The Explorer shows a project structure with 'SDLC' > 'CopilotLabs' > 'SS_Lab1' > 'lab1.py'. The editor shows the file 'lab1.py' with line 1: `user_name = input("Please enter your name: ")`. A tooltip is visible above the assignment, showing '< 1/1 > Accept [Tab] Accept Word [*] [→] ...'.

Step 2: Test Multi-line Function Suggestion



The screenshot shows the VS Code interface with the Explorer on the left and the editor on the right. The Explorer shows a project structure with 'SDLC' > 'CopilotLabs' > 'SS_Lab1' > 'lab1.py'. The editor shows the file 'lab1.py' with line 1: `user_name = input("Please enter your name: ")`. Line 3 shows the start of a function definition: `def greet_user(name, email, location):`. Line 4 shows the function body: `print(f"Hello, {name}!")`, `print(f"Your email is: {email}")`, and `print(f"You are located in: {location}")`.

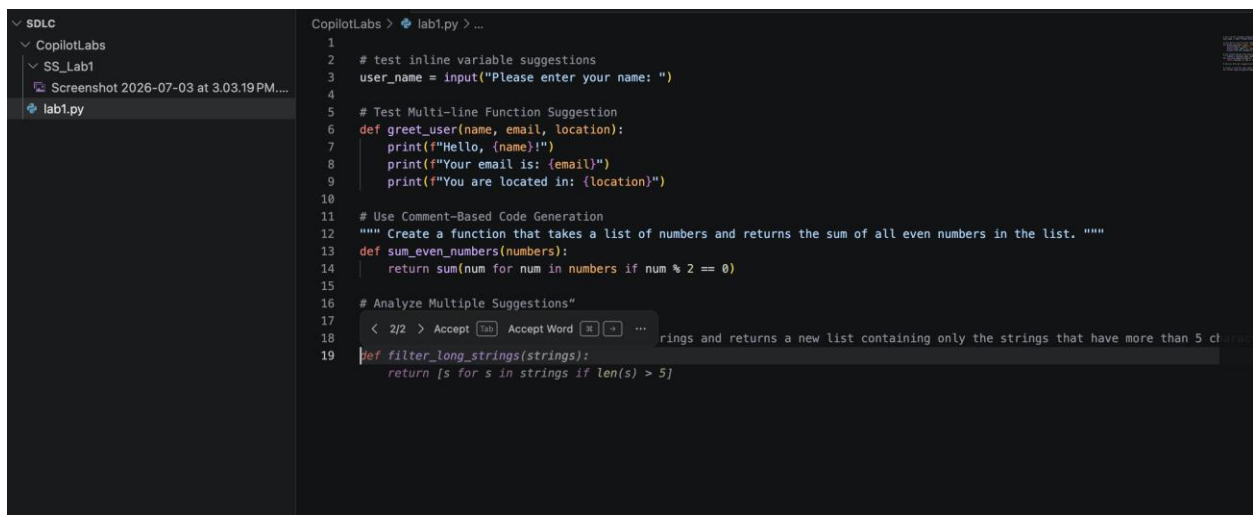
Step 3: Use Comment-Based Code Generation



The screenshot shows a code editor with the following text:

```
0
1  # Use Comment-Based Code Generation
2  """ Create a function that takes a list of numbers and
3  | returns the sum of all even numbers in the list."""
4  ✨
5  def sum_of_even_numbers(numbers):
6  →| return sum(num for num in numbers if num % 2 == 0)
```

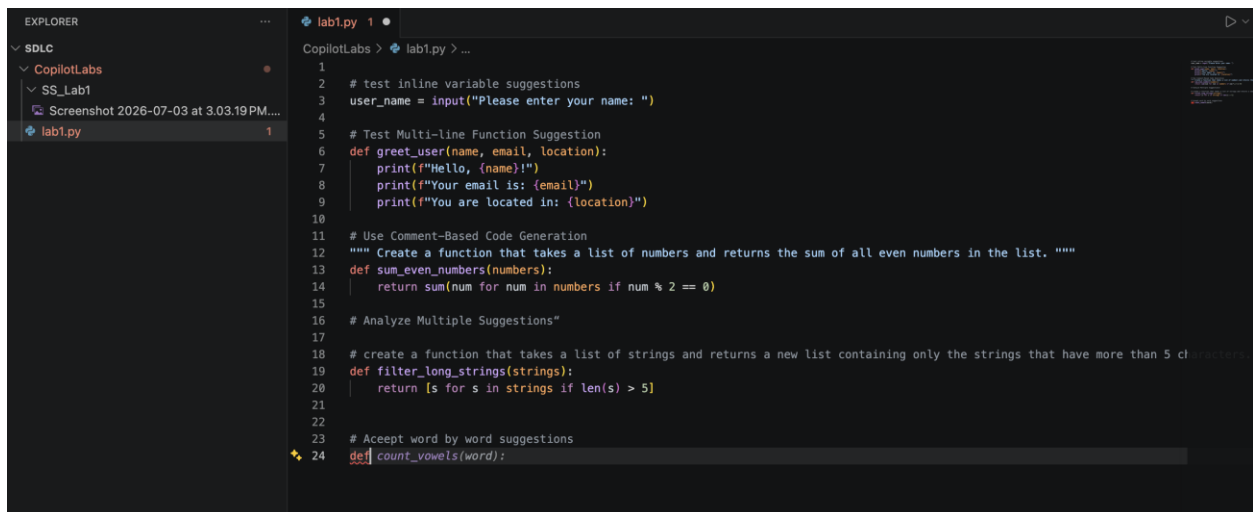
Step 4: Analyze Multiple Suggestions



The screenshot shows the VS Code editor with the Copilot Labs sidebar on the left. The main editor displays a Python file named lab1.py. A code suggestion popup is visible over the code, showing a multi-line function suggestion for a function named filter_long_strings. The popup includes a '2/2' indicator and buttons for 'Accept', 'Accept Word', and 'Cancel'.

```
1
2 # test inline variable suggestions
3 user_name = input("Please enter your name: ")
4
5 # Test Multi-line Function Suggestion
6 def greet_user(name, email, location):
7     print(f"Hello, {name}!")
8     print(f"Your email is: {email}")
9     print(f"You are located in: {location}")
10
11 # Use Comment-Based Code Generation
12 """ Create a function that takes a list of numbers and returns the sum of all even numbers in the list. """
13 def sum_even_numbers(numbers):
14     return sum(num for num in numbers if num % 2 == 0)
15
16 # Analyze Multiple Suggestions"
17 < 2/2 > Accept Accept Word Cancel ... rings and returns a new list containing only the strings that have more than 5 characters
18
19 def filter_long_strings(strings):
20     return [s for s in strings if len(s) > 5]
```

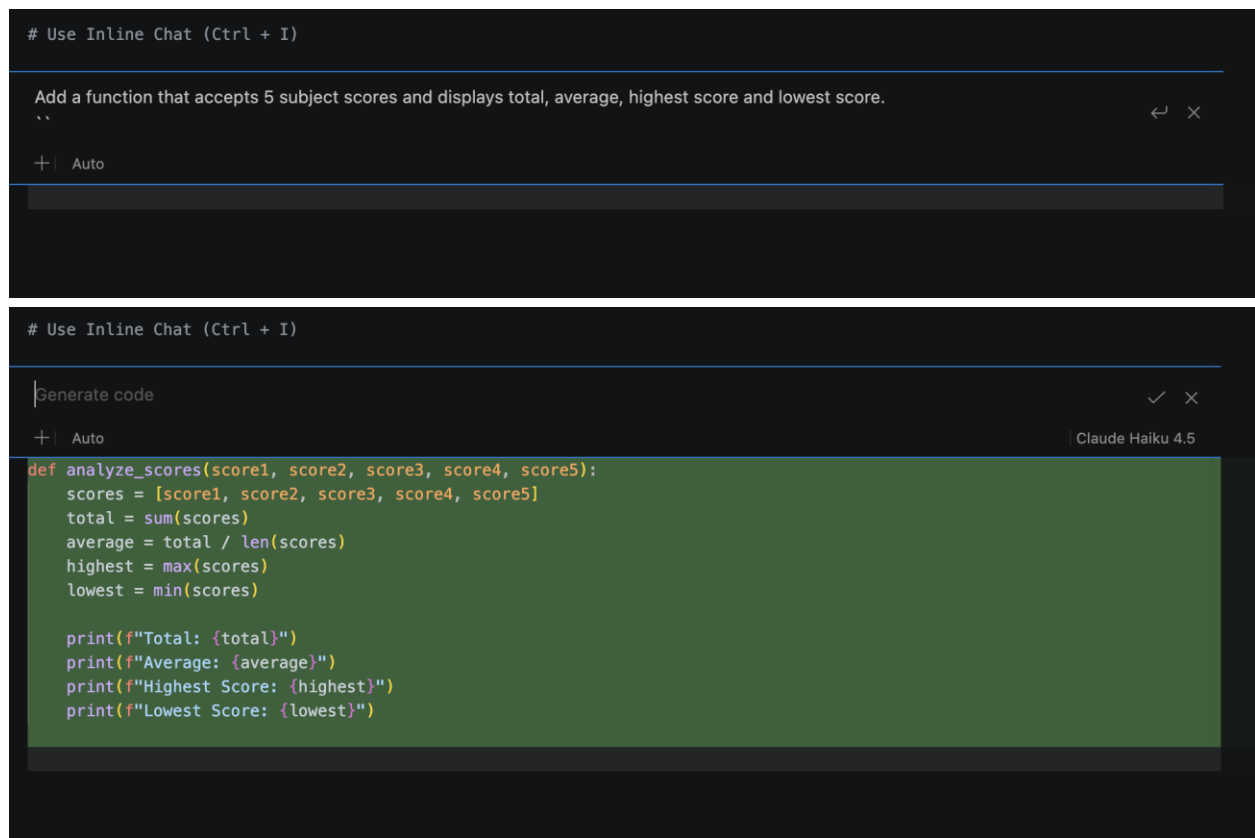
Step 5: Accept Word-by-Word



The screenshot shows the VS Code editor with the Copilot Labs sidebar on the left. The main editor displays the Python file lab1.py. The code is now complete, including the function filter_long_strings and a new function count_vowels. The Copilot Labs sidebar shows the file lab1.py with a '1' next to it, indicating one suggestion was accepted.

```
1
2 # test inline variable suggestions
3 user_name = input("Please enter your name: ")
4
5 # Test Multi-line Function Suggestion
6 def greet_user(name, email, location):
7     print(f"Hello, {name}!")
8     print(f"Your email is: {email}")
9     print(f"You are located in: {location}")
10
11 # Use Comment-Based Code Generation
12 """ Create a function that takes a list of numbers and returns the sum of all even numbers in the list. """
13 def sum_even_numbers(numbers):
14     return sum(num for num in numbers if num % 2 == 0)
15
16 # Analyze Multiple Suggestions"
17
18 # create a function that takes a list of strings and returns a new list containing only the strings that have more than 5 characters
19 def filter_long_strings(strings):
20     return [s for s in strings if len(s) > 5]
21
22
23 # Accept word by word suggestions
24 def count_vowels(word):
```

Step 6: Use Inline Chat (Ctrl + I)



Step 7: Use Copilot Chat Window

EXPLORER

SDLC

CopilotLabs

SS_Lab1

Screenshot 2026-07-03 at 3.03.19 PM....

lab1.py

lab1.py

CopilotLabs > lab1.py > ...

```
30 def analyze_scores(score1, score2, score3, score4, score5):
31     highest = max(scores)
32     lowest = min(scores)
33
34     print(f"Total: {total}")
35     print(f"Average: {average}")
36     print(f"Highest Score: {highest}")
37     print(f"Lowest Score: {lowest}")
38
39
40
41
42
43
44
45
46 def display_user_details(name, email, location, score1, score2, score3, score4, score5):
47     scores = [score1, score2, score3, score4, score5]
48     total = sum(scores)
49     average = total / len(scores)
50     highest = max(scores)
51     lowest = min(scores)
52
53     print(f"Name: {name}")
54     print(f>Email: {email}")
55     print(f"Location: {location}")
56     print(f"Total Score: {total}")
57     print(f"Average Score: {average}")
58     print(f"Highest Score: {highest}")
59     print(f"Lowest Score: {lowest}")
60
61
62
63 #
```

Keep Undo

CHAT

COMBINE USER DETAILS METHOD

Start Over

Add another method that combines and displays the user's name, email, location and score details together.

I'm checking the current Python file to find the class or function that owns those user fields, then I'll add the smallest method that formats them together.

Considered file inspection and reviewed lab1.py

The file only has standalone functions, not a class, so I'm adding a companion function beside the existing user helpers that prints the identity fields together with the computed score summary.

Edited • lab1.py +16 -0

I've made the local change. Next I'm running a quick syntax/error check on the touched file to confirm the new function fits cleanly with the existing script.

Reviewed lab1.py

Added a new function in CopilotLabs/lab1.py called

> 1 file changed +16 -0 Keep Undo

+ lab1.py

EXPLORER

SDLC

CopilotLabs

SS_Lab1

Screenshot 2026-07-03 at 3.03.19 PM....

lab1.py

lab1.py

CopilotLabs > lab1.py > ...

```
30 def analyze_scores(score1, score2, score3, score4, score5):
31     highest = max(scores)
32     lowest = min(scores)
33
34     print(f"Total: {total}")
35     print(f"Average: {average}")
36     print(f"Highest Score: {highest}")
37     print(f"Lowest Score: {lowest}")
38
39
40
41
42
43 # Using Copilot chat window to generate a function that takes user details and scores
44
45
46 def display_user_details(name, email, location, score1, score2, score3, score4, score5):
47     scores = [score1, score2, score3, score4, score5]
48     total = sum(scores)
49     average = total / len(scores)
50     highest = max(scores)
51     lowest = min(scores)
52
53     print(f"Name: {name}")
54     print(f>Email: {email}")
55     print(f"Location: {location}")
56     print(f"Total Score: {total}")
57     print(f"Average Score: {average}")
58     print(f"Highest Score: {highest}")
59     print(f"Lowest Score: {lowest}")
60
61
62
63
```

CHAT

COMBINE USER DETAILS METHOD

Add another method that combines and displays the user's name, email, location and score details together.

I'm checking the current Python file to find the class or function that owns those user fields, then I'll add the smallest method that formats them together.

Considered file inspection and reviewed lab1.py

The file only has standalone functions, not a class, so I'm adding a companion function beside the existing user helpers that prints the identity fields together with the computed score summary.

Edited • lab1.py +16 -0

I've made the local change. Next I'm running a quick syntax/error check on the touched file to confirm the new function fits cleanly with the existing script.

Reviewed lab1.py

Added a new function in CopilotLabs/lab1.py called

display_user_details(...) that combines

+ lab1.py

Describe what to build

+ | Auto

Local Default Approvals

